

Natural language processing

PDF to sentences

Finn Årup Nielsen

DTU Compute, Technical University of Denmark

September 10, 2026

PDF to sentences

Second session in course: Get a Web service running that extracts the sentences in a PDF file.

Details of the exercise is on DTU Learn under assignments.

Techniques for PDF extraction

Command line tools: For instance, `pdftotext` is a Linux program

Large general frameworks: `docling`, `MarkItDown`.

Dedicated extractors, e.g., `GROBID`.

Vision-language models, e.g., CampusAI's Gemma 4

Dedicated vision-language models, e.g., `PaddleOCR`, `Unlimited OCR`

There are old scientific papers that investigate the performance of PDF extractors: “A Benchmark and Evaluation for Text Extraction from PDF” (2017) and “A Benchmark of PDF Information Extraction Tools Using a Multi-task and Multi-domain Evaluation Framework for Academic Documents” (2023). New: “Benchmarking Document Parsers on Mathematical Formula Extraction from PDFs” (2026)

GROBID

GROBID with homepage at <https://grobid.org/>

GROBID is targeted at extracting from scientific papers.

GROBID is a Java program, but:

GROBID is available as a Docker container with a trained deep learning model.

The output from GROBID may be in the special “TEI” XML format where XPath can be used to extract relevant elements, e.g., by etree in the lxml package.

It can also perform sentence tokenization.

Vision-language models

Some openai-compatible LLM API interfaces can handle images (vision-language models).

You might need to convert the file to the data URL format, e.g., with `pymupdf`:

```
1 import pymupdf
2
3 doc = pymupdf.open(PDF_FILENAME)
4 page = doc[0] # First page
5 pix = page.get_pixmap()
6 png_bytes = pix.tobytes("png")
7 encoded = base64.b64encode(png_bytes).decode("ascii")
8 data_url = f"data:image/png;base64,{encoded}"
```

You can control the resolution with the `dpi` parameter of the `get_pixmap` method.

FastAPI and file input

You can POST files to a FastAPI Web service.

Example asynchronous endpoint

```
1 from fastapi import FastAPI, File, UploadFile, HTTPException
2
3 @app.post("/v1/extract-sentences")
4 async def extract_sentences(pdf_file: UploadFile = File(...)):
5     content = await pdf_file.read()
6     # ...
```

Multiple Containers

If you use the GROBID container then you will need another container to implement the interface that returns the sentences in the appropriate format.

You can use Docker Compose and a `compose.yaml` to start the two containers.

Sentence tokenization

Quick and dirty implementation with a simple(?) regular expression

```
1 text = ""Dr. Smith went to Washington D.C. last week. He said,  
2 'Hello!' and left. It was amazing... isn't it?""  
3  
4 # Split on punctuation followed by whitespace  
5 import re  
6 sentences = re.split(r'(?<=[.!?])\s+', text)
```

?<= is the “positive lookbehind assertion”. Result:

```
1 ['Dr.', 'Smith went to Washington D.C.', 'last week.',  
2 "He said,\n'Hello!' and left.", 'It was amazing...', "isn't it?"]
```


Sentence tokenization with NLTK

NLTK with a one-time `nltk.download('punkt_tab')`

```
1 import nltk
2
3 sentences = nltk.sent_tokenize(text)
```

Result:

```
1 ['Dr. Smith went to Washington D.C. last week.', "He said,\n'Hello!'",
2  'and left.', "It was amazing... isn't it?"]
```

But with Danish

```
1 >>> nltk.sent_tokenize("F.eks. 3.0 er en score.")
2 ['F.eks.', '3.0 er en score.']
```

Sentence tokenization with spaCy

After spacy download en_core_web_sm in the terminal. The sentences is available via sentence tokenization in the .sents attribute

```
1 import spacy
2
3 nlp = spacy.load("en_core_web_sm")
4 doc = nlp(text)
5 sentences = [sent.text.strip() for sent in doc.sents]
```

Result

```
1 ['Dr. Smith went to Washington D.C. last week.',
2  "He said,\n'Hello!' and left.", "It was amazing... isn't it?"]
```

Danish (note it is an English model):

```
1 >>> doc = nlp("F.eks. 3.0 er en score.")
2 >>> [sent.text.strip() for sent in doc.sents]
3 ['F.eks. 3.0 er en score.']
```

Sentence tokenization with CampusAI

```
1 import os
2 from dotenv import load_dotenv
3 from openai import OpenAI
4
5 load_dotenv()
6 client = OpenAI(api_key=os.environ["CAMPUSAI_API_KEY"],
7                 base_url=os.environ["CAMPUSAI_API_URL"])
8
9 r = client.chat.completions.create(
10     model=os.environ["CAMPUSAI_MODEL"],
11     messages=[{"role": "user",
12                "content": f"Split into sentences. Return one sentence per line.\n{
13 text}"},
14 ]),
15 sentences = r.choices[0].message.content.splitlines()
```

Result

```
1 ['Dr. Smith went to Washington D.C. last week.',
2  "He said, 'Hello!' and left.", "It was amazing... isn't it?"]
```

Could there be a problem with long text when using CampusAI/LLM-based?

Setting up data in a docker container

Dockerfile entry so set up a data file for NLTK

```
1 ENV NLTK_DATA=/usr/local/share/nltk_data  
2 RUN python -m nltk.downloader --dir $NLTK_DATA punkt_tab
```

The directory with the data file is one of those examined, see `nltk.data.path`.

Question

What kind of tool should convert the PDF to text?

What tool should be used to convert the text to sentences

If a GROBID container is used, should synchronous or asynchronous communication be used to that container.

What are difficult conversions?